Set Operations Using Bit Strings

Union, intersection and difference of sets over the universal set U = {1,2,3,4,5,6,7,8,9}.

1. Algorithm

Idea: The universal set U = {1,...,9} is fixed in an array. Any subset is stored as a bit string where bit k = 1 means U[k] is in the subset. Set operations become bitwise operations.

1. **Start.**
2. **Define** the universal set `uset[] = {1,2,3,4,5,6,7,8,9}` and `SIZE = 9`.
3. **Initialise** bit strings A, B, C with all bits 0.
4. **Read set A:** read m. For each of the m inputs x, find k such that `uset[k] == x` and set `A[k] = 1`. If x is not found, ignore it with a message.
5. **Read set B** the same way into B.
6. **Union:** for k = 0 to SIZE-1, `C[k] = A[k] | B[k]`. Print C.
7. **Intersection:** for k = 0 to SIZE-1, `C[k] = A[k] & B[k]`. Print C.
8. **Difference (A - B):** for k = 0 to SIZE-1, `C[k] = A[k] && !B[k]`. Print C.
9. **Print a set:** for k = 0 to SIZE-1, if bit[k] = 1 print `uset[k]`, separated by commas, inside braces.
10. **Stop.**

2. Code

```
#include <stdio.h>

#define SIZE 9

typedef struct { unsigned char x : 1; } bit;

/* universal set (hardcoded) */
const int uset[SIZE] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

void readset(bit a[], int n);
void printset(const bit a[]);
void unionset(const bit a[], const bit b[], bit c[]);
void intersect(const bit a[], const bit b[], bit c[]);
void difference(const bit a[], const bit b[], bit c[]);

int main(void)
{
    bit a[SIZE] = {{0}}, b[SIZE] = {{0}}, c[SIZE] = {{0}};
    int n;

    printf("Universal set: {1,2,3,4,5,6,7,8,9}\n\n");

    printf("No. of elements in set A: ");
    if (scanf("%d", &n) != 1) return 1;
    readset(a, n);

    printf("No. of elements in set B: ");
    if (scanf("%d", &n) != 1) return 1;
    readset(b, n);

    printf("\nSet A: ");
    printset(a);
    printf("Set B: ");
    printset(b);

    unionset(a, b, c);
    printf("\n\tA U B = ");
    printset(c);

    intersect(a, b, c);
    printf("\n\tA intersect B = ");
    printset(c);

    difference(a, b, c);
    printf("\n\tA - B = ");
    printset(c);

    return 0;
}

/* Read n elements of a subset and store as a bit string */
void readset(bit a[], int n)
{
    int i, k, x;

    if (n < 0 || n > SIZE) {
        printf("Invalid size; must be between 0 and %d.\n", SIZE);
        return;
```

```
    }

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) {
        if (scanf("%d", &x) != 1) return;
        for (k = 0; k < SIZE; k++) {
            if (uset[k] == x) { a[k].x = 1; break; }
        }
        if (k == SIZE)
            printf("%d is not in the universal set, ignored.\n", x);
    }
}

/* Print a set from its bit string representation */
void printset(const bit a[])
{
    int k, first = 1;
    printf("{");
    for (k = 0; k < SIZE; k++) {
        if (a[k].x) {
            printf(first ? "%d" : ",%d", uset[k]);
            first = 0;
        }
    }
    printf("}\n");
}

void unionset(const bit a[], const bit b[], bit c[])
{
    for (int k = 0; k < SIZE; k++)
        c[k].x = a[k].x | b[k].x;
}

void intersect(const bit a[], const bit b[], bit c[])
{
    for (int k = 0; k < SIZE; k++)
        c[k].x = a[k].x & b[k].x;
}

void difference(const bit a[], const bit b[], bit c[])
{
    for (int k = 0; k < SIZE; k++)
        c[k].x = a[k].x && !b[k].x;
}
```

3. Output

Sample run

```
Universal set: {1,2,3,4,5,6,7,8,9}

No. of elements in set A: 4
Enter 4 elements: 1 2 3 4
No. of elements in set B: 4
Enter 4 elements: 3 4 5 6

Set A: {1,2,3,4}
Set B: {3,4,5,6}

    A U B = {1,2,3,4,5,6}

    A intersect B = {3,4}

    A - B = {1,2}
```

Edge case: element not in universal set

```
No. of elements in set A: 3
Enter 3 elements: 1 2 99
99 is not in the universal set, ignored.
No. of elements in set B: 2
Enter 2 elements: 3 4

Set A: {1,2}
Set B: {3,4}
```

Edge case: empty result

```
Set A: {1,2}
Set B: {3,4}

    A intersect B = {}
```